

Sprite & Group

the OOP layer over pygame primitives

From raw rects and manual loops → one update(), one draw(), N objects.

Where Day 8 left us

Yesterday: a bouncing ball drawn directly with `pygame.draw.circle`. Position, velocity, and bounds-checks all lived in the main loop as plain variables.

DAY 8 — raw primitives

```
# state lives as loose variables
x, y = 100, 100
vx, vy = 4, 3

while running:
    # update
    x += vx; y += vy
    if x < 0 or x > W: vx = -vx

    # draw
    pygame.draw.circle(screen, ...)
```

DAY 9 — Sprite + Group

```
# state lives inside the object
balls = pygame.sprite.Group()
for _ in range(50):
    balls.add(Ball())

while running:
    balls.update() # all 50, once
    balls.draw(screen) # all 50, once

# loop body stays small no matter
# how many objects you add
```

What is a Sprite?

`pygame.sprite.Sprite` is a base class for any visible game object — a ball, a paddle, a brick. You subclass it, give it an image and a rect, and pygame knows how to manage it inside a Group.

```
import pygame

class Ball(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.Surface((20, 20))
        self.image.fill((255, 80, 80))
        self.rect = self.image.get_rect()
        self.rect.center = (320, 240)

    def update(self):
        # called once per frame by the group
        self.rect.x += 2
```

ANATOMY

`super().__init__()`

Required — wires this object into pygame's internal sprite registry.

`self.image`

The Surface that gets blitted. Can be a colored Surface or a loaded PNG.

`self.rect`

A Rect that holds position and size. Move the sprite by mutating this.

`update(self)`

Per-frame behavior. The Group calls this on every member.

Moving state into the object

Each ball owns its own velocity. The bounce check becomes a method on the class, not loose code in the main loop. Now every ball runs the same logic independently.

```
class Ball(pygame.sprite.Sprite):
    def __init__(self, x, y):
        super().__init__()
        self.image = pygame.Surface((20, 20), pygame.SRCALPHA)
        pygame.draw.circle(self.image, (80, 180, 255), (10, 10), 10)
        self.rect = self.image.get_rect(center=(x, y))
        self.vx = random.choice([-4, 4])
        self.vy = random.choice([-3, 3])

    def update(self):
        self.rect.x += self.vx
        self.rect.y += self.vy
        if self.rect.left < 0 or self.rect.right > 640:
            self.vx = -self.vx
        if self.rect.top < 0 or self.rect.bottom > 480:
            self.vy = -self.vy
```

WHAT CHANGED

Constructor takes (x, y)

Spawn each ball at a different position by passing it in.

vx, vy live on the instance

Each ball has its own velocity. No shared global state.

Bounce moved into update()

Wall-hit logic belongs to the ball, not the main loop.

SRCALPHA for transparency

Lets the circle blit cleanly without a black square around it.

Group: one call, N updates

`pygame.sprite.Group` is a container for sprites. Calling `group.update()` runs `update()` on every member. Calling `group.draw(screen)` blits every member's image at its rect. No manual loop.

```
balls = pygame.sprite.Group()

for _ in range(50):
    x = random.randint(20, 620)
    y = random.randint(20, 460)
    balls.add(Ball(x, y))

while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    balls.update()          # physics for all 50
    screen.fill((15,20,40))
    balls.draw(screen)      # render all 50
    pygame.display.flip()
```

KEY OPERATIONS

`group.add(sprite)`

Register a sprite for management.

`group.update()`

Calls `.update()` on every member, in add-order.

`group.draw(surface)`

Blits every member's image at its rect.

`sprite.kill()`

Removes the sprite from every group it belongs to.

Scaling from 1 ball to 50

The point of the Sprite/Group pattern is invisible scaling. Once the class exists, the cost of going from 1 object to 50 is one loop. The main loop never gets longer.

```
import pygame, random
from ball import Ball

pygame.init()
screen = pygame.display.set_mode((640, 480))
clock = pygame.time.Clock()

balls = pygame.sprite.Group()
for _ in range(50):
    balls.add(Ball(
        random.randint(20, 620),
        random.randint(20, 460)))

running = True
while running:
    clock.tick(60)
    balls.update()
    balls.draw(screen)
```

WHY THIS SCALES

Loop body is constant

1 ball or 1000 balls — the while loop has the same 4 lines.

State is isolated

Each ball's vx/vy is its own. No risk of one ball's data leaking into another's.

Easy to delete

sprite.kill() removes it from the group — Day 10's brick wall will use this.

Raw primitives vs Sprite & Group

Both render the same scene. The Sprite version is longer for 1 object but shorter for N — and dramatically easier to extend.

DIMENSION	RAW PRIMITIVES	SPRITE + GROUP	WINNER
Lines for 1 object	~10 — short and direct	~25 — class boilerplate	Raw
Lines for 50 objects	~60 — manual lists	~28 — one loop	Sprite
Adding behavior	Edit main loop	Add method to class	Sprite
Removing one object	Manual list bookkeeping	sprite.kill()	Sprite

 TAKEAWAY

The boilerplate buys you scaling. Day 10 collisions and Day 13 Pong both lean hard on Group — adopt the pattern now.

Refactor + populate

Take Day 8's bouncing-ball script. Refactor the ball into a Sprite subclass. Spawn 50 of them at random positions with random velocities into a single Group. SPACE still pauses.

STRUCTURE

ball.py — Ball(Sprite)
main.py — game loop
All physics inside update()
Use SRCALPHA for the circle
Window: 640 × 480, 60 FPS

CLASSES

Ball(pygame.sprite.Sprite)
__init__(x, y) — random vx, vy
self.image — colored Surface
self.rect — get_rect(center=...)
update() — move + bounce

BEHAVIOR

Spawn 50 balls into a Group
group.update() + group.draw()
Walls: flip vx/vy on contact
SPACE toggles a paused flag
ESC or window-close exits

STRETCH: vary each ball's size and color randomly. Verify each ball still owns its own state.

Watch for these

⚠ **MUST call `super().__init__()`**

If you skip it, pygame won't register the sprite. `group.add()` will silently fail to manage it, and you'll see nothing on screen with no error.

⚠ **`self.image` AND `self.rect` required**

`Group.draw(screen)` reads both. Missing either → `AttributeError` when the group tries to render. Set them in `__init__` before anything else uses the sprite.

💡 **Rect coords are integers**

`rect.x = 1.5` silently truncates to 1. For sub-pixel motion, keep float position on the sprite (`self.fx`, `self.fy`), then assign `rect.x = int(self.fx)` each frame.

💡 **Group order = draw order**

Sprites added later draw on top. If your paddle hides behind balls, add the paddle to its group after the balls — or use separate groups and draw in the order you want.

Where Sprite + Group goes next

DAY 10

Wed May 20

Collisions

`spritecollide()` and `groupcollide()` work directly on the Groups you build today. Brick-breaker prototype: paddle + ball + wall of bricks.

DAY 11

Thu May 21

Sound + animation

Frame-based animation cycles through a list of `self.image` surfaces. Same Sprite class, image swapped per frame.

DAY 13

Sat May 23

Pong (proper)

Two paddles + ball, all Sprites. Scoring, paddle-angle deflection, simple AI. End of Week 2 deliverable.

Every game object from here on is a Sprite. The pattern is non-negotiable now.